



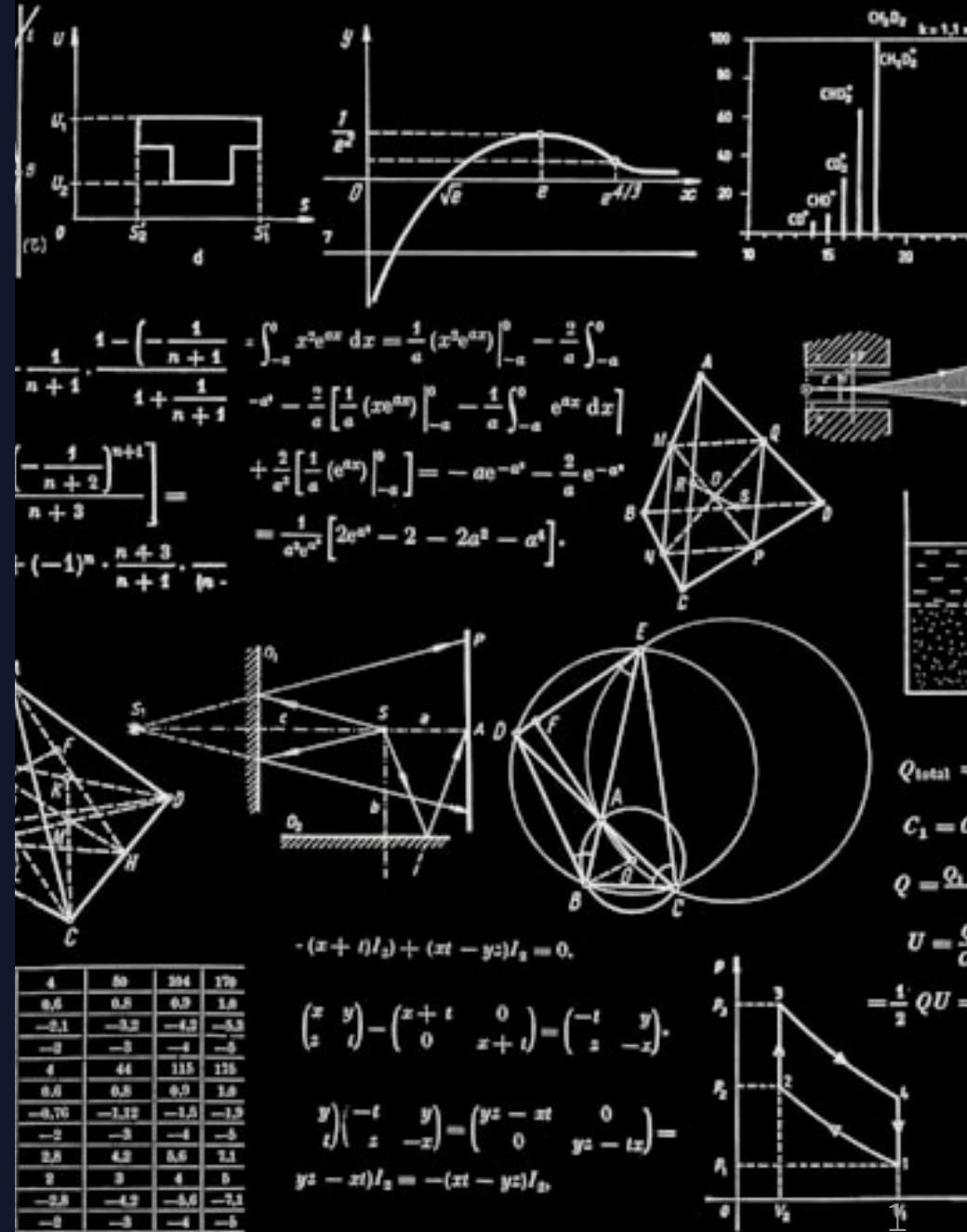
L22:

Inheritance

Security Droid Upgrade

Neon City Cyber Academy

Module 5: Object-Oriented Programming





The Upgrade Problem

Current: Worker Droids can greet and work

Need: Security Droids that can ALSO patrol

Bad Solution: Copy all Worker code + add patrol

Good Solution: INHERIT from Worker, ADD new features!



What is Inheritance?

Inheritance = Child class gets ALL features of parent class

Real World:

-  **Parent:** Human
-  **Child:** Baby (inherits eyes, hair, etc.)
- **+** **Extra:** Learns to code (new skill!)

Code: SecurityDroid inherits from Droid + adds patrol()



Creating a Child Class

```
class Droid:
    def __init__(self, name, battery_level):
        self.name = name
        self.battery_level = battery_level

    def greet(self):
        print(f"Greetings. I am {self.name}, Worker Unit.")

class SecurityDroid(Droid):
    pass # Inherits everything!
```

Syntax: `class Child(Parent):`



What Gets Inherited?

```
security = SecurityDroid('GUARD-X1', 100)
security.greet() # Works! Inherited from Droid
```

SecurityDroid automatically has:

- ☒ `__init__` method
- ☒ `name` and `battery_level` attributes
- ☒ `greet()` method

WITHOUT writing any code!

+ Adding New Methods

```
class SecurityDroid(Droid):  
    def patrol(self):  
        print(f"[{self.name}] Initiating sector patrol. All clear.")
```

```
guard = SecurityDroid('SENTINEL-9', 98)  
guard.greet()    # Inherited method  
guard.patrol()   # New method!
```

Child has parent features PLUS its own!



Parent vs Child Capabilities

```
worker = Droid('WORKER-5', 100)
guard = SecurityDroid('GUARD-5', 100)

# Worker can only greet
worker.greet() # ✓ Works
# worker.patrol() # ✗ Error! Workers can't patrol

# Guard can do BOTH
guard.greet() # ✓ Inherited
guard.patrol() # ✓ Unique to SecurityDroid
```

Inheritance = Code reuse without copy-paste!



Inheritance Hierarchy

```
Droid (Parent)
├── greet()
├── __init__()
└── SecurityDroid (Child)
    ├── greet() [inherited]
    ├── __init__() [inherited]
    └── patrol() [new]
```

SecurityDroid IS-A Droid (with extras!)



Multiple Child Classes

```
class Droid:
    # Base class

class SecurityDroid(Droid):
    def patrol(self):
        print("Patrolling...")

class MedicDroid(Droid):
    def heal(self):
        print("Healing unit...")

class CombatDroid(Droid):
    def attack(self):
        print("Engaging target!")
```



When to Use Inheritance

Use when:

- Child "IS-A" type of Parent
- Need to add features to existing code
- Want to reuse code without copying

Examples:

- Dog IS-A Animal
- SecurityDroid IS-A Droid
- ElectricCar IS-A Car



Quick Reference

```
# Define parent
```

```
class Parent:  
    def method1(self):  
        pass
```

```
# Child inherits from Parent
```

```
class Child(Parent):  
    def method2(self):  
        pass
```

```
# Create instances
```

```
p = Parent()  
p.method1() # ✓
```

```
c = Child()  
c.method1() # ✓ Inherited  
c.method2() # ✓ Own method
```

✓ Lesson Complete!

You Mastered:

- ✓ What inheritance is (child from parent)
- ✓ Syntax: `class Child(Parent):`
- ✓ Child automatically gets parent methods
- ✓ Adding new methods to child class
- ✓ IS-A relationship
- ✓ Multiple children from one parent

Next: L23 - Advanced Inheritance



Challenge

Create a Droid Hierarchy:

- Base: **Droid**
- Children: **WorkerDroid**, **SecurityDroid**, **CommanderDroid**
- Each child adds 2 unique methods
- Create instances of each type

Build a complete droid army!